Java FTP Client Project Report

Dao Hoang Minh Triet
Student ID: 10423179
Vietnamese-German University

Abstract

This report presents a Java FTP client application with a simple JavaFX graphical user interface. The application supports the main FTP operations needed for everyday remote file management: connecting to a server, logging in, viewing the current directory, changing directories, listing files through passive mode, uploading, downloading, deleting files, creating directories, removing directories, and quitting the session. The design separates the protocol implementation from the GUI controller, so the networking logic remains easy to understand and test. The report explains the application architecture, FTP protocol flow, passive data connection handling, reply parsing, and the relationship between the GUI and FTP commands.

Index Terms

FTP client, Java, JavaFX, socket programming, passive mode, file transfer.

I. Introduction

The goal of the application is to provide a small FTP client in Java that can communicate with a public or local FTP server. The client uses the FTP control connection for commands and a separate data connection for operations such as listing, uploading, and downloading files. The protocol part is implemented mainly with `java.io.*`, `java.net.*`, and `java.util.*`, while JavaFX is used for the graphical interface.

The implemented application is a lightweight FTP client with a clear GUI. The user enters the host, port, username, and password, then clicks *Connect*. After login, the remote directory is shown in a list view. The right side of the interface provides buttons for common FTP actions, including creating directories, deleting files, removing directories, uploading files, and downloading files. A console at the bottom shows the FTP protocol conversation, which is useful for debugging and for demonstrating that real FTP commands are sent to the server.

II. Application Features

The FTP client is designed as a practical remote file management tool. Table I summarizes the main features implemented in the application.

TABLE I
Application Feature Summary

Feature	Implementation
Connect to FTP server	`FtpClient.connect()` opens the control socket on port 21 or the selected port.
Login	`login()` sends `USER` and `PASS`.
PWD and CWD	`printWorkingDirectory()` and `changeDirectory()` implement `PWD` and `CWD`.
LIST	`list()` calls `PASV`, opens a data socket, then sends `LIST`.
GET and PUT	`downloadFile()` uses `PASV` and `RETR`; `uploadFile()` uses `PASV` and `STOR`.
Remote operations	`DELE`, `MKD`, and `RMD` are implemented.
Quit	`quit()` sends `QUIT` and closes resources.
GUI	JavaFX view and controller provide a simple graphical interface.
Reply parsing	`readReply()` supports normal and multiline FTP replies.
Resource management	Sockets and streams are closed using `try`-with-resources or explicit close methods.

III. Application Architecture

The project is organized around two main responsibilities. First, `FtpClient` implements the FTP protocol using sockets and streams. Second, the JavaFX classes provide the interface and call the FTP client methods when the user clicks a button. This separation keeps the network code independent from the GUI code.

The main classes are listed below:

- `Launcher`: starts the JavaFX application.
- `Application`: loads the GUI layout and creates the main window.
- `Controller`: handles button events, list view navigation, file choosers, and console output.
- `FtpClient`: manages FTP commands, replies, sockets, passive mode, upload, and download.
- `FtpReply`: stores a numeric FTP reply code and its message.
- `FtpReplyCodes`: defines constants for common FTP reply codes such as 220, 230, 227, 150, 226, and 550.

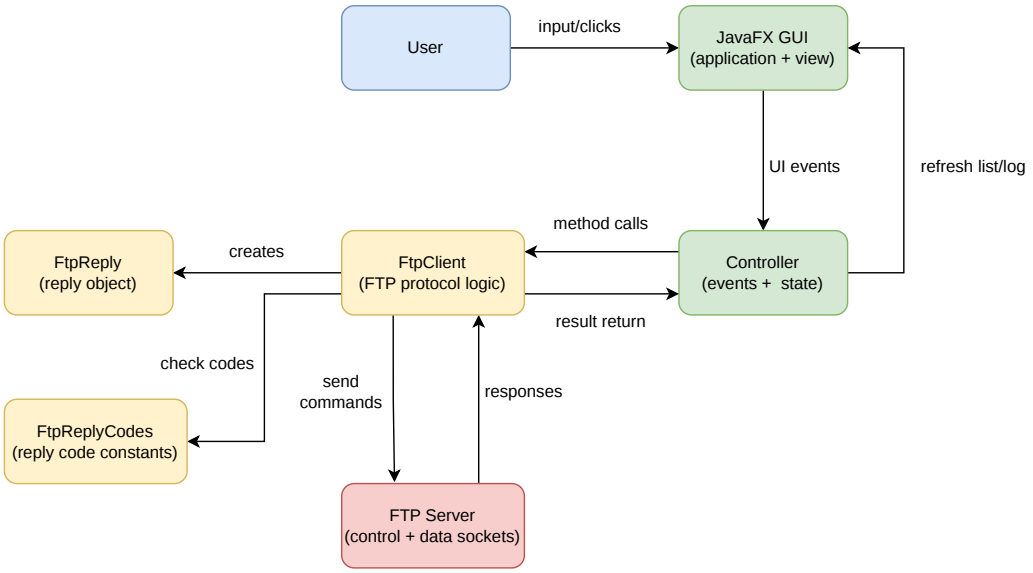

Fig. 1. High-level architecture of the FTP client.

Figure 1 shows that the GUI does not directly manage sockets. Instead, it calls high-level methods such as `list()`, `uploadFile()`, and `downloadFile()`. The FTP client class then sends the required FTP commands and reads the server replies.

IV. FTP Protocol Design

FTP uses two types of connections. The first is the control connection, which remains open during the session and carries commands such as `USER`, `PASS`, `PWD`, and `QUIT`. The second is the data connection, which is opened separately for file listings and file transfers. This project uses passive mode because it is easier for clients behind NAT or firewalls: the client asks the server for a data endpoint using `PASV`, then connects to the returned address and port.

A. Connection and Login Flow

When connecting, the client opens a socket to the server and waits for reply code 220, meaning the service is ready. It then sends `USER`. If the server returns 331, the client sends `PASS`. Login succeeds when the server returns 230.

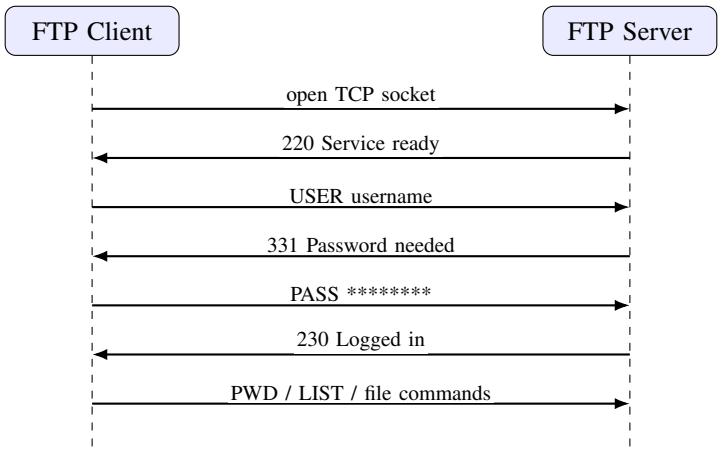

Fig. 2. Connection and login sequence.

B. Passive Listing and Transfer Flow

The most important part of the project is using a separate passive data connection. For `LIST`, `RETR`, and `STOR`, the client first sends `PASV`. The server returns reply code 227 with six numbers. The first four numbers represent the IP address, and the last two numbers represent the port using the formula:

$$port = p_1 \times 256 + p_2. \tag{1}$$

After calculating the port, the client opens the data socket. Then it sends the transfer command on the control connection. The file bytes or listing text are exchanged through the data connection. Finally, the server sends a completion reply such as 226 on the control connection.

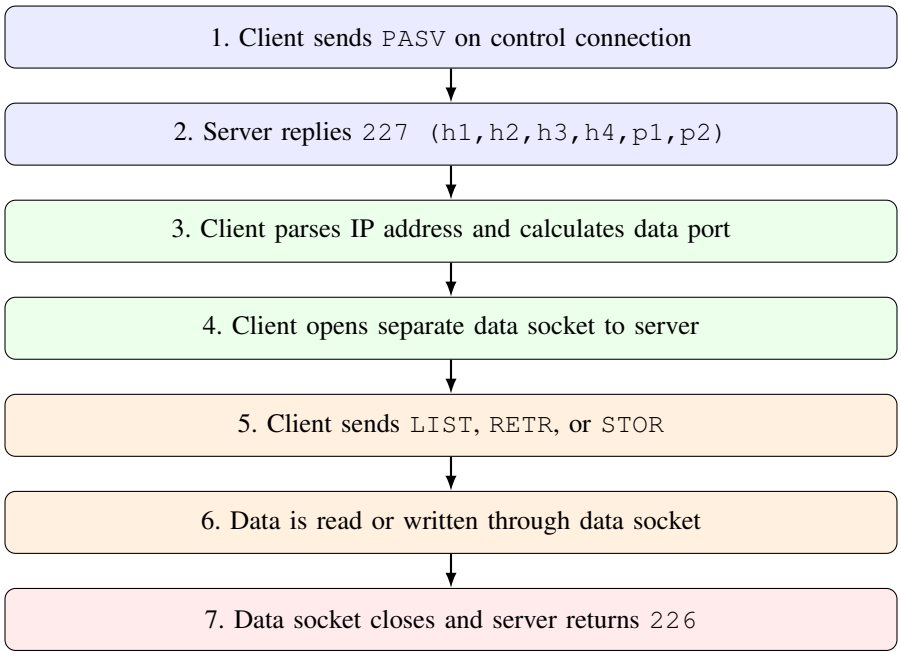

Fig. 3. Passive mode flow for listing, downloading, and uploading.

V. Implementation Details

A. Control Connection and Commands

The control connection is created in `FtpClient.connect()`. The client stores a `Socket`, a `BufferedReader`, and a `BufferedWriter`. Commands are sent using CRLF line endings, as required by FTP. The method

`sendCommand()` also writes the command to the GUI protocol log. For security in the display, the password is masked as `PASS ********`.

The class provides one method for each major operation. For example, `changeDirectory()` sends `CWD`, `createDirectory()` sends `MKD`, `deleteFile()` sends `DELE`, and `removeDirectory()` sends `RMD`. Each method checks the returned FTP reply code and throws an `IOException` when the server reports an error. This makes errors easy for the GUI to catch and display.

B. Reply Parsing

FTP servers can return single-line or multiline replies. The method `readReply()` first reads the initial line and extracts the three-digit reply code. If the fourth character is a hyphen, the method continues reading until it finds a line that starts with the same code followed by a space. This correctly handles replies such as:

```
220-First welcome line
More server information
220 Last welcome line
```

The parsed result is stored in a `FtpReply` object containing the numeric code and the full message.

C. Upload and Download

Before upload and download, the client switches to binary mode using `TYPE I`. This is important because binary mode avoids unwanted text conversion and works for documents, images, archives, and other file types. Download uses `RETR`: the client reads bytes from the data socket and writes them into a local file. Upload uses `STOR`: the client reads bytes from a local file and writes them into the data socket. A 4096-byte buffer is used for the transfer loop.

D. Resource Management and Error Handling

The code uses `try`-with-resources for data sockets and file streams in listing, uploading, and downloading. The control connection is closed by `quit()` and `close()`. If an operation fails, the protocol class throws `IOException`; the controller catches it and prints a readable message in the console. This keeps the application stable and prevents unused sockets or streams from staying open after an operation finishes.

VI. Graphical User Interface

The GUI is simple and practical. It contains input fields for host, port, username, and password. The center area shows the current remote directory listing, and the right side contains action buttons. The bottom console displays the FTP command and reply history. Figure 4 shows the current application screen.

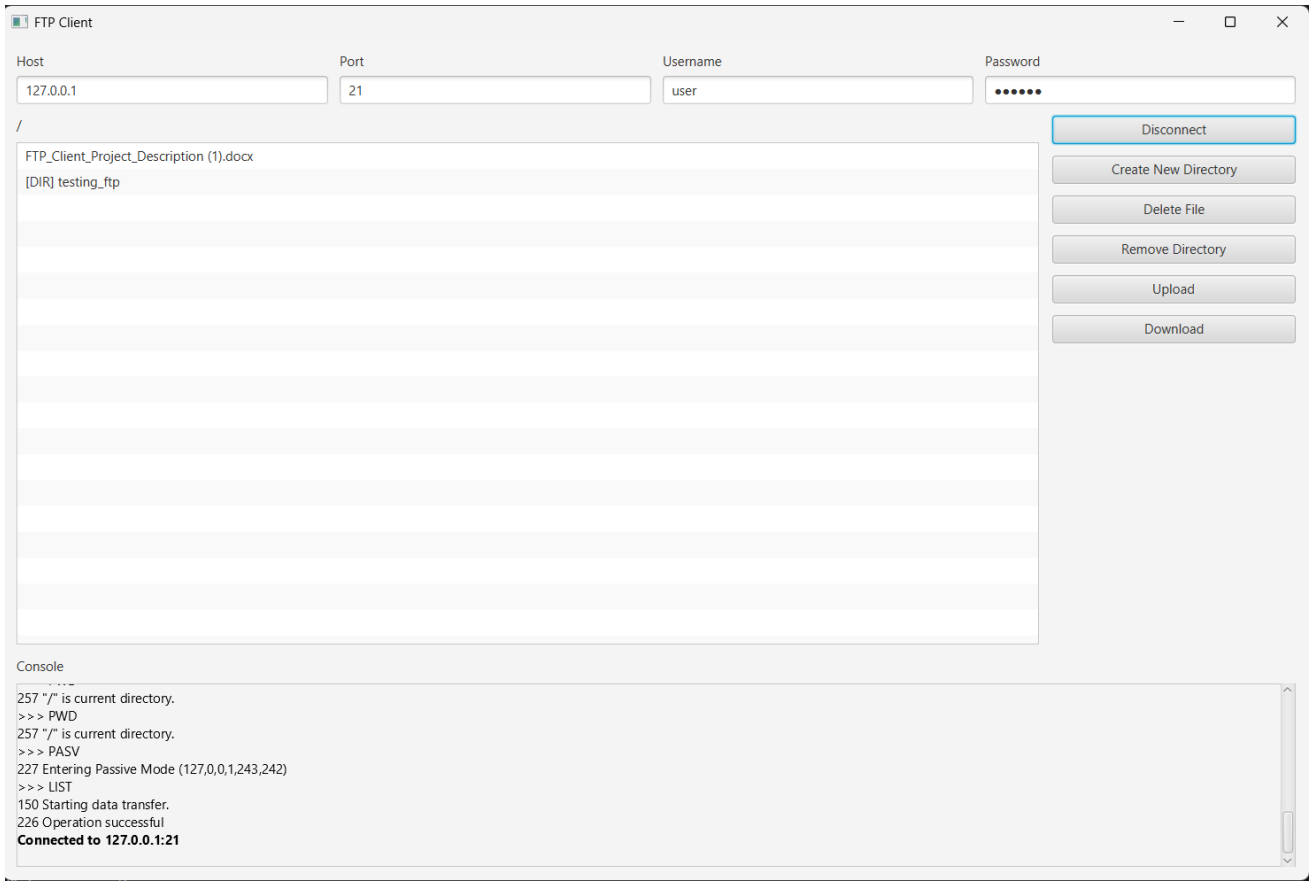

Fig. 4. FTP client graphical user interface.

The controller keeps track of whether the application is connected. When disconnected, file operation buttons are disabled. After a successful login, the controller calls `PWD` and `LIST`, updates the path label, and fills the list view. Directories are shown with a `[DIR]` prefix. Double-clicking a directory sends `CWD` and refreshes the list. Upload and download use JavaFX file chooser dialogs.

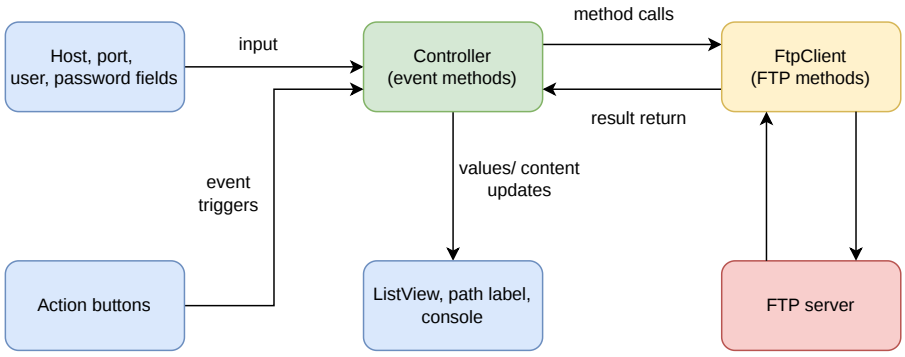

Fig. 5. GUI event handling structure.

VII. Example Test Commands

During testing, the GUI console shows real FTP commands and replies. A typical test session contains the following sequence:

```
220 220-FileZilla Server 1.12.6
220 Please visit https://filezilla-project.org/
```

```
>> USER user
331 331 Please, specify the password.

>> PASS ********
230 230 Login successful.

>> PWD
257 257 "/" is current directory.

>> PASV
227 227 Entering Passive Mode (127,0,0,1,239,188)

>> LIST
150 150 Starting data transfer.

226 226 Operation successful
```

Additional tests can be performed by creating a directory, uploading a small file, downloading it again, deleting the file, removing the directory, and finally disconnecting with `QUIT`. These tests cover the main application functions: login, PWD, CWD, PASV+LIST, PASV+RETR, PASV+STOR, DELE, MKD, RMD, QUIT, reply parsing, and resource management.

VIII. Limitations and Future Improvements

The current implementation is intentionally simple and focuses on the core FTP commands. It supports plain FTP, not FTPS. Some modern servers require TLS before accepting passwords, so those servers will reject this client. Another limitation is that GUI operations are currently executed synchronously. For very large transfers or slow servers, the interface may temporarily freeze. A future version could move network operations into background JavaFX tasks and show progress bars for upload and download.

IX. Conclusion

This project implements a functional Java FTP client with a simple GUI and a manually written FTP protocol layer. The application supports connection, login, directory navigation, passive listing, binary upload, binary download, deletion, directory creation, directory removal, multiline reply parsing, and resource cleanup. The design is easy to understand because the GUI controller is separated from the protocol implementation. Overall, the project demonstrates socket programming, FTP command flow, passive data connections, and practical Java GUI development.